

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Name: Kushaal Shyam Potta

Reg. No: 3122235001071

Exp. No: 5

Date: 07-08-2026

Implementation of Syntax Checker using Lex and Yacc Tools

Aim:

To build a syntax checker with Lex and Yacc that recognizes declaration, assignment, conditional and looping statements, using a suitable grammar for each. It works on C++ source: apart from the C-like statements it accepts the bool type, cout << and cin >> statements, return, #include and using namespace lines, and an int main() wrapper.

Tools Used:

- Flex, the lexical analyzer generator
- Bison, the parser generator (Yacc)
- GCC

Code:

syntax.l

```
%{
#include "y.tab.h"
%}

%option yylineno noyywrap nounput noinput

DIGIT  [0-9]
LETTER [a-zA-Z_]

%%

"//"*. *           ;
"/"*(\[^\*]\|\\*\[^\*/\])*\*+/"          ;

^\[ \t]*"#\[^\n]*           ;
"using"[ \t]+"namespace"[ \t]+{LETTER}({LETTER}|{DIGIT})*[ \t]*";"    ;

"int"|"float"|"char"|"double"|"bool" { return TYPE; }
"return"                             { return RETURN; }
"std::"?"cout"                       { return COUT; }
"std::"?"cin"                        { return CIN; }
"std::"?"endl"                       { return ENDL; }
"true"|"false"                      { return NUM; }
"if"                                 { return IF; }
"else"                               { return ELSE; }
"while"                              { return WHILE; }
"do"                                 { return DO; }
"for"                                { return FOR; }

{LETTER}({LETTER}|{DIGIT})*         { return ID; }
{DIGIT}+(\.{DIGIT})?                { return NUM; }
\"([^\\"\\n]|\\.)*\"                 { return STR; }
\'([^\'\\]|\\.)*\'                   { return NUM; }

"++"                                { return INC; }
"--"                                { return DEC; }
"<<"                                { return SHL; }
">>"                                { return SHR; }
```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

"&&"      { return AND; }
"||"      { return OR; }
"=="|"!=" { return EQOP; }
"<="|">="|"<"|>" { return RELOP; }
"! "      { return NOT; }
"="       { return '='; }
[+\-*/%]  { return yytext[0]; }
[(){};,]  { return yytext[0]; }

[ \t\r\n] ;
.         { return yytext[0]; }

%%

```

syntax.y

```

%{
#include <stdio.h>

int yylex(void);
void yyerror(const char *s);

extern int  yylineno;
extern char *yytext;
%}

%token TYPE IF ELSE WHILE DO FOR ID NUM STR
%token RETURN COUT CIN ENDL SHL SHR
%token INC DEC AND OR EQOP RELOP NOT

%left OR
%left AND
%left EQOP
%left RELOP
%left '+' '-'
%left '*' '/' '%'
%right NOT UMINUS

%nonassoc LOWER_THAN_ELSE
%nonassoc ELSE

%%

program      : stmt_list
              | function
              ;

/* int main() { ... }      (optionally int main(void) ) */
function     : TYPE ID '(' ')' block
              | TYPE ID '(' TYPE ')' block
              ;

stmt_list    : stmt_list stmt
              | stmt
              ;

stmt         : decl_stmt
              | assign_stmt
              | if_stmt
              | while_stmt
              | do_stmt
              | for_stmt
              | io_stmt
              | return_stmt
              | block
              | ';'

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

;

/* Output / input statements: cout << a << "text" << endl; cin >> a; */
io_stmt : COUT out_list ';'
        | CIN in_list ';'
        ;

out_list : SHL out_item
        | out_list SHL out_item
        ;

out_item : STR
        | ENDL
        | expr
        ;

in_list : SHR ID
        | in_list SHR ID
        ;

return_stmt : RETURN expr ';'
            | RETURN ';'
            ;

block : '{' stmt_list '}'
      | '{' '}'
      ;

/* Declaration statement: int i, a = 5; */
decl_stmt : TYPE decl_list ';'
          ;

decl_list : decl_list ',' declarator
          | declarator
          ;

declarator : ID
          | ID '=' expr
          ;

/* Assignment statement: i = i + 1; i++; */
assign_stmt : simple_stmt ';'
            ;

simple_stmt : ID '=' expr
           | ID INC
           | ID DEC
           ;

/* Conditional statements: if, if-else */
if_stmt : IF '(' expr ')' stmt %prec LOWER_THAN_ELSE
        | IF '(' expr ')' stmt ELSE stmt
        ;

/* Looping statements: while, do-while, for */
while_stmt : WHILE '(' expr ')' stmt
           ;

do_stmt : DO stmt WHILE '(' expr ')' ';'
        ;

for_stmt : FOR '(' simple_stmt ';' expr ';' simple_stmt ')' stmt
        ;

expr : expr OR expr
     | expr AND expr
     | expr EQOP expr

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

| expr RELOP expr
| expr '+' expr
| expr '-' expr
| expr '*' expr
| expr '/' expr
| expr '%' expr
| NOT expr
| '-' expr %prec UMINUS
| '(' expr ')'
| ID
| NUM
;

%%

void yyerror(const char *s) {
    printf("Syntax Error at line %d near '%s'\n", yylineno, yytext);
}

int main(void) {
    if (yyparse() == 0)
        printf("Syntactically correct\n");
    return 0;
}

```

Input (input.txt)

```

int i, a;
i = 0;
a=5;
while (i < 10)
{
    if ( i<a)
    i = i + 1;
    else
    i = i - 1;
}

```

Commands Used

```

lex syntax.l
yacc -d syntax.y
cc lex.yy.c y.tab.c -lfl -o syncheck
./syncheck < input.txt

```

Output:

```

● kushaal@Kushaals-MacBook-Air Assignment 5 % flex syntax.l
● kushaal@Kushaals-MacBook-Air Assignment 5 % yacc -d syntax.y
● kushaal@Kushaals-MacBook-Air Assignment 5 % gcc lex.yy.c y.tab.c -o syncheck
● kushaal@Kushaals-MacBook-Air Assignment 5 % ./syncheck < input.txt
Syntactically correct

```

Result for the given input

The grammar has no conflicts: operator precedence and associativity are set with %left / %right and the dangling else is settled with %prec LOWER_THAN_ELSE, so yacc reports no shift/reduce conflicts.

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Additional Test Cases:

```

● kushaal@Kushaals-MacBook-Air Assignment 5 % cat input2.txt
int n = 5, sum, i;
float avg;
sum = 0;
for (i = 0; i < n; i++)
{
    if (i % 2 == 0 && i != 4)
        sum = sum + i * 2;
    else if (!(i > 3))
        sum = sum - 1;
}
do
{
    n--;
} while (n >= 0 || sum > 100);
avg = sum / (n + 1);
● kushaal@Kushaals-MacBook-Air Assignment 5 % ./syncheck < input2.txt
Syntactically correct
● kushaal@Kushaals-MacBook-Air Assignment 5 % cat input3.txt
int i, a;
i = 0
a = 5;
while (i < 10)
{
    i = i + 1;
}
● kushaal@Kushaals-MacBook-Air Assignment 5 % ./syncheck < input3.txt
Syntax Error at line 3 near 'a'
● kushaal@Kushaals-MacBook-Air Assignment 5 % cat input4.txt
int i;
i = 0;
while (i < 10
{
    i = i + 1;
}
● kushaal@Kushaals-MacBook-Air Assignment 5 % ./syncheck < input4.txt
Syntax Error at line 4 near '{'

```

Further test cases: one valid program and two with syntax errors

input2.txt contains every kind of statement: declarations with initial values, for, if / else if, do-while, logical operators and increment / decrement. It is accepted as Syntactically correct.

input3.txt is missing the semicolon after `i = 0`, and input4.txt is missing the closing bracket of the while condition. The checker prints the line number and the token at which parsing can no longer continue.

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

C++ test cases (input5.txt and input6.txt):

```

input5.txt:
#include <iostream>
using namespace std;

int main()
{
    int n = 5, sum = 0, i;
    bool done = false;
    cin >> n;
    for (i = 0; i < n; i++)
    {
        if (i % 2 == 0 && i != 4)
            sum = sum + i * 2;
        else
            sum = sum - 1;
    }
    cout << "Sum = " << sum << endl;
    return 0;
}
Result: Syntactically correct

input6.txt (missing semicolon after endl):
#include <iostream>
using namespace std;

int main()
{
    int a = 5;
    cout << "a is " << a << endl
    return 0;
}
Result: Syntax Error at line 8 near 'return'

```

Learning Outcomes:

- Learned to write a grammar for declaration, assignment, conditional (if-else) and looping (while, do-while, for) statements of a C / C++ like language.
- Understood how precedence declarations (%left, %right) and %prec eliminate shift/reduce conflicts, including the dangling else.
- Understood how Yacc generates a parser from BNF-style productions and reports syntax errors through yyerror, with yylineno giving the line number.
- Gained experience combining Lex tokens with a Yacc grammar to validate the structure of a small program.